Chapter 1

Model framework

1.1 Model framework: introduction

Apols for broken section references: if I compile only one chapter, I can't keep the correct refs, but there's no other way to produce single chunks of writing (that I know of.)

The model framework implements the key economic and space concepts from chapter **??** and provides a structure for agents using those implementations to interact. It is built using object-oriented programming, as discussed in section **??**. Relations between objects can be used the describe the logic of the model if they are divested of unnecessary detail. (Appendix **??** describes the code more formally.)

Table 1.1 gives a key for the diagrams that will be used to do this. The rationale for this approach is to present the object structure and the logic of the algorithms in one diagram as clearly as possible, but they should be used in conjunction with the text description. One coding norm that will be used here is to capitalise objects, when they are being discussed as objects. For instance, 'Good' is a particular object, distinct from goods generally.

1.1.1 Chapter overview

The chapter covers the following topics:

- The rest of this section describes the three elements that form the foundation for the model framework: actor setup, decision structure and actor data. These are outlined briefly and then discussed in more detail in the following section.
- This section also covers the units used in the model framework
- The next section describes Actor setup for Firms and People in detail
- The model 'scenario' description method is outlined

- The chapter ends with a discussion and critique

Element	Description
Variable Parameter Constant	Grey boxes are values. Variables change in model runs. Parameters are fixed for model runs but may change between them Constants do not change
Object array	An array, or set, of objects, such as the set of all firms
Reference to object	For example, a Person may want to keep a reference to their preferred employer
interface	A way to provide shared methods to different objects; not important for understanding model logic, but worth noting where present
method	An object method for achieving a particular model aim
logic	Can be one of three things: a logical decision, a calculation Or an operation on a set (array)
Actor object	People or firms
Non-actor object	Any other object used in the framework

Table 1.1: Key for model diagrams

1.1.2 Elements

Two actor setups

The two actor setups are:

- Single actor type only
- Two actor types: People and Firms

The single actor setup requires all actors to decide who else to interact with. No exogenous distinction between Firms and People is built into the model structure. Two-type models, however, do impose that distinction: Firms and People have separate decision structures and motivations.

This draft sticks with explaining the People-Firm model. I'll need to explain the single actor version somewhere. I'm getting the most complex one done first before worrying about that.

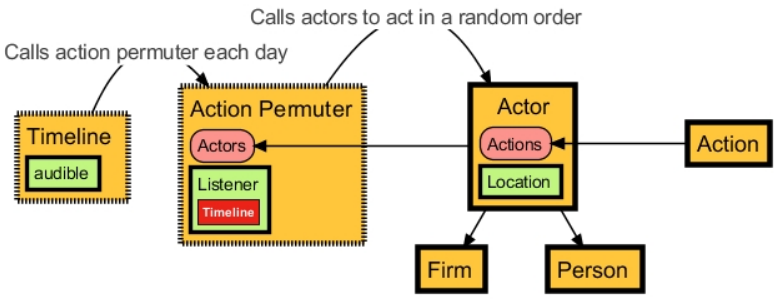


Figure 1.1: Timeline sequence

Decision structures and model timeline

The timing structure shown in figure 1.1 is very simple: as each day passes on the 'Timeline', an 'Action Permuter' calls all actors in a random order.[1] When an Actor is called, they decide what action to take. Actors' decisions are based on one or both of the following:

- Instant: act on information collected 'now'
- Feedback: Act, wait, respond

As discussed in section **??**, actors take decisions either instantaneously or using feedback from their actions over one or more timesteps. In instant decisions, when actors are called to act, they immediately gather the information they need and optimise based on that. For multi-timestep decisions, actors try an action and then check later whether the outcome was desirable or not before taking their next action. No attempt is made to simulate synchronous decision-making. At the point where an actor is deciding, all others are currently static.

more on async vs sync decision-making

Separation of timings for different agent types. Needs covering. Here?

Two actor data sources

Actor decisions will be based on one or both of the following sources of information:

- Internal actor data, changed by other actors since last a decision was taken. (Example: firm net income.)
- Externally collected data (Example: list of good prices from surrounding firms.)

Externally collected data can come only from other Actors. This is acquired by searching space for a list of all appropriate Actors and forming a set from it.

[1] The Timeline is part of an Observer framework used by other model elements like visualisation and data writing; again, see the code appendix for more details.

These three elements - Actor setup, decision structure and Actor data - form the foundation for the model framework. Section 1.2 explains how they are put together. Because all actors face a static world at the point where they decide, their actions can be logically described separately.

Units

The thesis models are abstract: they are not calibrated to any particular real-world situation. As far as possible, then, values are normalised to keep analysis and presentation as clear as possible. That said, it is still worth discussing the way numbers are set up in the model.

Some values represent excludable, rivalrous values, where only one actor can 'have' them. Both stock and money can come into this category, but note that in a particular model 'scenario' (see below), the 'flow' of these values may be turned off. For example, if the wage and goods prices are both fixed, money flow may also be turned off.

In the models that use explicit space, distance measurement is built around the idea of one Person's 'day'. The distance a Person can get in one day serves as one unit. That distance Actors can get in one day may change over a model run, but it still stands as a base for comparison in that run. Distances will be described this way:

- A given distance between r and s will be labelled rs. The time is takes to cover it is t_{rs}. So, generically, $t_{rs} = f(rs, tech)$, where $tech$ is some technology that has an impact on the time taken. For many of the models, time to cover a given distance will be a function of the distance alone. However, the generic factor $tech$ here indicates that the amount of time to cover a distance may change.

- $tech$'s role in this function is separate from its cost. For example, if $t_{rs} = rs/tech$, the time taken to travel between r and s goes down as the level of $tech$ increases. It says nothing about $tech$'s cost. Indeed, from the point of view of actors, it could be free and ubiquitous.

Since each Person has one day to spend, the number of People determines the total amount of available labour in any model scenario. If People are also spending that day on movement, there is thus a zero-sum relationship between time contributed to production versus space.

Lastly, the nominal value of wage and good price matters less than their ratio: what a Person can buy with their wage. This is the 'real wage'.

1.2 Actor setup

1.2.1 How People act

People have 'one day per day' to spend and, as section **??** pointed out, their fundamental question is, 'what is the maximum utility I can get for my time?' This time can be spent on moving through space, on production in their own backyard, or put into firms or collaborators

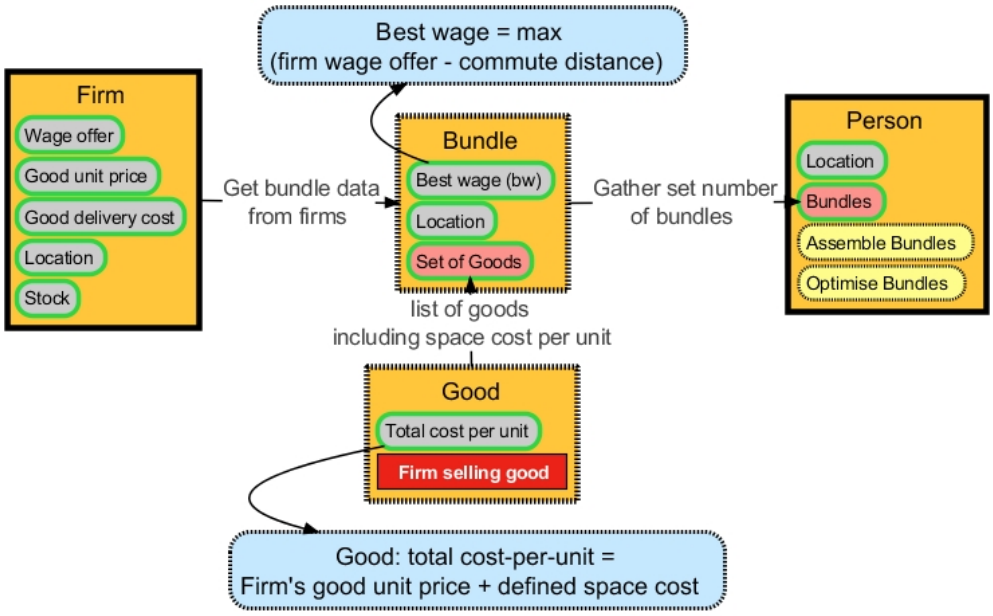


Figure 1.2: Elements for the Person action

in exchange for either goods or money. The description here deals with the most complex of these: people working for money and then spending it on goods.

People compare a number of 'Bundles'. Each Bundle consists of a location, a best wage offer and a list of goods. They then choose the one that will give them the highest utility.

Figures 1.2 and 1.3 outline what happens when a Person is called to act. An 'instant' decision is made on what to buy, who to work for and where to locate based on data they gather when it is their turn. Figure 1.2 shows all the elements prior to utility maximisation. As well as their set of Bundles, People have two methods: 'assemble Bundles' and 'optimise Bundles'. The next section (and figure 1.3) describe this last step in more detail. An overview of the methods involved is as follows:

- Method 'assemble Bundles': assemble a series of Bundles; one from my current location, and a defined number from other random locations.
- Method 'optimise Bundles': two steps.
  - Optimise each bundle's mix of goods to get a single utility value
  - Select the Bundle with the best utility: accept the wage, pay for goods and, if applicable, move to the new location.

Assembling Bundles

As outlined in figure 1.2, a Bundle consists of the following:

- The centre location of the Bundle. One of these will centre on the Person's current location. Others are given random locations within a defined range. The rationale here is this: each separate Bundle allows a Person to ask, 'if I were to move to that spot, would I get a better utility value?'
- A set of Goods, where each Good is an object containing the good's total cost per unit (including delivery cost) and a reference to the Firm selling the good. That reference provides a way to check stock levels as well as to buy goods from Firms if the Bundle is selected.
- A 'best wage' selected from a set of Firms and taking account of commute cost.

Each of these is dealt with next, but here are a few preliminaries. Firstly, What determines the number of random bundles? The value is arbitrary: the higher it is, the more likely on any one turn that a person will find the 'best' utility option for them. Over a number of days, however, this will tend to happen anyway. Section

Section on sensitivity testing for timing structures

goes into the issue in more depth: what difference do timing structures make for outcomes? The range for a Bundle's centre location can also vary. It may be limited by one day's distance, weighted towards closer distances, or People may be given free choice over the whole model space.

The first step for a person getting Bundle data is to get two lists of Firms: one for finding the best wage, another for a full list of available goods. Two sets are required since distance to potential employers may be restricted by commute distance, whereas good delivery can occur from further afield.

Getting a set of Goods

A Bundle will contain a list of all goods, either within a given range or for the whole model space. Each good is represented by a 'Good' object that contains the following:

- Cost per unit, including delivery cost.
- Reference to the Firm selling the good.

As regards good delivery costs, there are various ways of calculating this. See section x

Section for outlining approaches to pricing goods across space

for full details, but an example illustrates the principle. If a firm has a base unit price p and a delivery cost to move one unit for one 'mile' d_p, and the distance to deliver is af, then the cost per unit can be set to $p + (p \cdot d_p \cdot af)$. This is the value stored in the Good object. The fact that a space cost is part of that is hidden to the Person: they make their decision based entirely on this single calculated cost.

Selecting the best wage

What I'd like to do here: use the set of Firm objects, and be allowed to define values within those objects, such as the firm wage offer, and location, so I can use one set notation line for saying what I mean. Options? Anyway - here's my attempt. If we go with this, I'll need to outline it above.

Also: zero sum time for commute or work - this is a movement-buying factor. Again, though, I'm just writing up the model as it stands and will work out where to put the MBF stuff (if at all) later.

Each Bundle has one unique 'best wage' offer. Selecting this is a two-step process: first, gather a set of all Firms within commute distance. Second, select the Firm offering the best wage, accounting for commute distance from the Bundle's location centre. This best wage can be described as follows. Let $F = \{f_1, f_2, ..., f_n\}$ be the set of Firm objects. Each Firm has a wage offer, which is 'the amount paid for one day's input'. For each $f \in F$, W_f is that wage offer. To describe space cost, the distance in *time* can be used: for each $f \in F$, t_{fa} is the time taken to travel between f and a.

Let l_{af} be the labour available to contribute to f by Person a. The amount of labour they have available for contributing is just their own time (one day, $t_a = 1$) minus commute time. Here, a 'commute' is taken to be twice t_{af}. The upper distance limit on whether a firm enters into a's considerations has to be less than a's one available day in total - half a day there, half back - otherwise there is no time left at all to contribute to production. So f can only be a maximum of half a day away. Given all this -

$$l_{af} = t_a - 2 \cdot t_{af},\ 0 < l_a < 1$$

If W_f is f's wage for a full day, and a has l_{af} to contribute (a fraction of their full day), this means the wage they will get is just $W_f \cdot l_{af}$. The best wage for Person p can now be defined as:

$$bw_p = max\ \{W_f \cdot l_{af} \mid W_f \in F,\ t_{af} < 0.5\} \tag{1.1}$$

So, the best wage for Person a is the maximum value from the following set: all wage offers proportionate to Person a's labour input from those firms in F who are less than half a day away.

Now, was that overkill? Could I just have used this last sentence?

Optimising bundles

Once a Bundle is assembled, it can be passed to the Person's optimisation routine.[2] This produces a utility level, which is stored in the Bundle. Finally, the Bundle with the maximum utility level is selected: the wage offer is accepted and the optimal quantity of goods is bought.

[2] As appendix **??** explains, there is only one actual, static Optimiser class, but in terms of 'functional isomorphism' it can be considered internal to each Person.

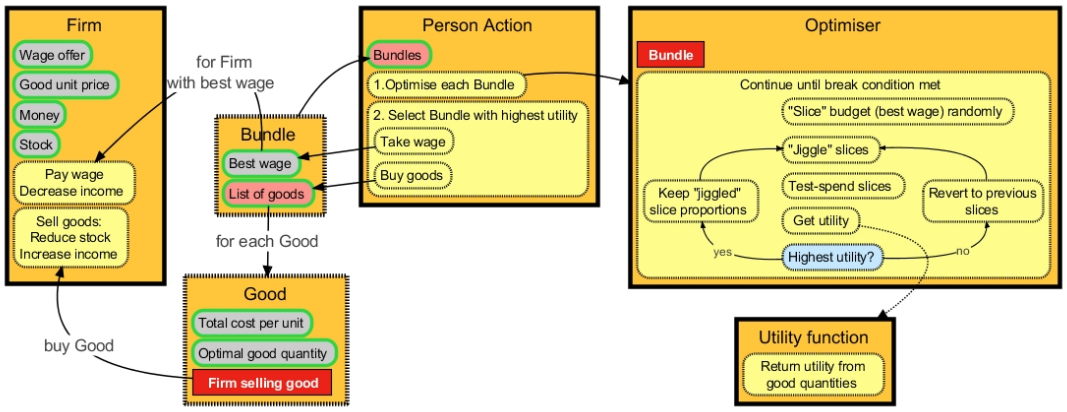


Figure 1.3: Optimisation of Bundles in a Person's action

The logical goal is simple: for each Bundle, find out how to spend the best wage on the set of Goods as optimally as possible (without actually spending it) then choose the optimal Bundle. Figure 1.3 is an overview of the process. Assuming the Person has the set of Bundles they will optimise, the sequence is as follows.

- Optimise each Bundle. This happens in an Optimiser:
  - The Optimiser iterates, either for a set number or until some condition is met (such as no marginal utility gain for a certain number of steps: see below.)
  - The iteration is initialised by randomly slicing the budget (the best wage), one slice for each Good in the bundle, before beginning the iterations. The iterations are:
  - 'Jiggle' the slices, changing their proportionate size slightly.
  - 'Test-spend' those slices on each Good: find out how much could be bought with the current budget slices using the per-unit price for the Good.
  - Get a utility level for these quantities of goods. The Utility function is logically separate: the Optimiser deals only with this process of random slicing and adjusting.
  - If the utility level found is the highest so far, keep the current proportion of slices and return to the 'jiggle' point. Else, revert to the previous slice proportions and try again. This has the effect of ratchetting the budget slices (and thus the spend per Good) towards an optimum.
- Once each Bundle has an optimum utility, select the Bundle with the highest:
  - Take the best wage: this instructs the Firm to pay the Person.
  - Buy goods in the quantities found optimal by this Bundle.

Figure 1.4 graphs data from this process in action. A bundle containing sixteen goods of equal cost is optimised for a CES function over a thousand iterations. (The optimum for a

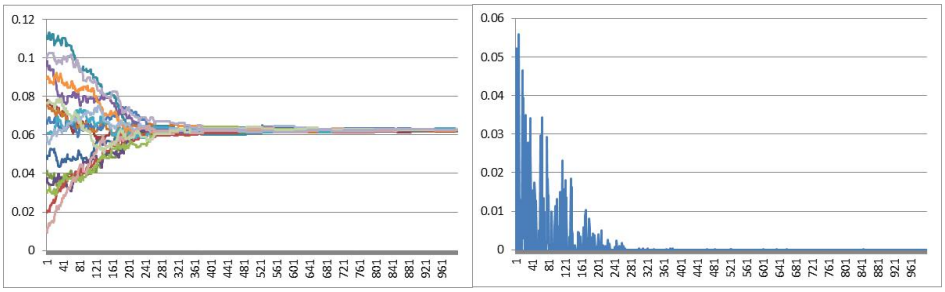


Figure 1.4: Utility-maximising stochastic search algorithm, optimising a CES function with 16 goods over a thousand iterations (x axis)

CES function with equal-value goods is already known from section **??**: an even mix.) In this example, the optimum is found quite slowly: this helps to illustrate the process. The first graph shows initially random budget split guesses converging on the optimum mix of 0.06 for each good. The second shows the marginal utility gain from each iteration; most of the gains happen very quickly, after which almost no extra benefit accrues (though there is a very small amount of further convergence.)

The important difference to analytic constrained optimisation is that people can work with any number and type of good and can deal with limited stock levels.

Need to illustrate what happens in limited stock situations, what happens to money if it can't all be spent.

In contrast, a standard constrained optimisation results in an 'objective' optimum (see **??**). In this model framework, people may find that this optimal amount simply isn't available to them, so they must spend their wage accordingly. As well as being able to deal with any quantity of available good, this approach also allows easy swapping of utility functions: there is no need to produce separate constrained optimisation equations for different situations.

somewhere: need to compare this limited notion of rationality with the critiqued 'superhuman' rationality that conflates the ability to tell better from worse with perfect knowledge of the whole market. Also, the 'realism' of 1000 iterations for finding utility. Here probably isn't the place. Also x-ref to Fowler's agent model, which uses objective optima, and Firms can use that signal.

Overview of People's utility

The technical detail of optimising utility has just been covered. There are three further points worth covering briefly.

People have utility functions based on those described in section **??**. In any one model run, all actors have identical tastes: they all use the same utility function. One useful assumption from using identical tastes is that, in one-good models, only the absolute quantity of that good matters. This is because the same quantity of good will always return the same utility for each

Person, regardless of the form of utility. For example, if all people have an identical one-good utility function that happens to have increasing returns, the increasing returns will just produce the same level of utility for everyone, given the same good input. In contrast, if tastes were not identical, one Person could gain more utility from the same quantity of good as another. Section **??** discusses a one-good situation where the raw-good amount stands in for utility.

In all situations where more than one good is available, People use utility functions. The form of utility used must also imply the production structure, though not the actual production function itself. For instance, a two-good utility function means a model where only two goods are made, though many firms may compete to sell each good. An n-good utility scenario, using the **Constant Elasticity of Substitution (CES)** function, will have each firm making one unique good, as they do in **monopolistic competition** models.

Actors using this method are rational in only the most basic way: they are able to judge when one Bundle is better than another. This rationality reflects basic micro-economic preference assumptions. Bundles of goods are 'complete': any two bundles can be compared. They are also transitive: as Varian puts it, "if the consumer thinks that X is at least as good as Y and that Y is at least as good as Z, then the consumer thinks that X is at least as good as Z" (Varian 2006 p.35).

1.2.2 How Firms act

A Firm's decision process is based on feedback between timesteps. A firm takes an action, waits, and then responds depending on how its own internal data has changed (see figure 1.5). Because of this, the structure of Firm decision-making is quite a lot easier to describe, since Firms do not collect any data externally themselves.

At least in the simplest version they don't. They might at some point: may be the only way to solve coordination?

At time $t-1$, a firm chooses one of either location, good unit cost or wage offer, and 'jiggles' it, altering it slightly in a random direction. It remembers these choices. At time t the firm asks, 'did my net income between timesteps increase?' If yes, it repeats the previous action (hence needing to remember what it was), with the possible addition of increasing the size of the adjustment. If not, the Firm returns to its initial random search.

again: this is their current method. Given I want to address coordination, this will be added to. Has been already, but needs more work.

The result is that, through blind search, Firms are constantly searching for ways to improve their net income. Note that this method disconnects them even further from any analytic approach to optimising. Where at least People do use a utility function (albeit one that calculates utility solely from raw good amounts), here the firm does not directly consider its own input or output requirements. Also, while they act on how their own internal data changes, Firms know nothing about how those changes occur.

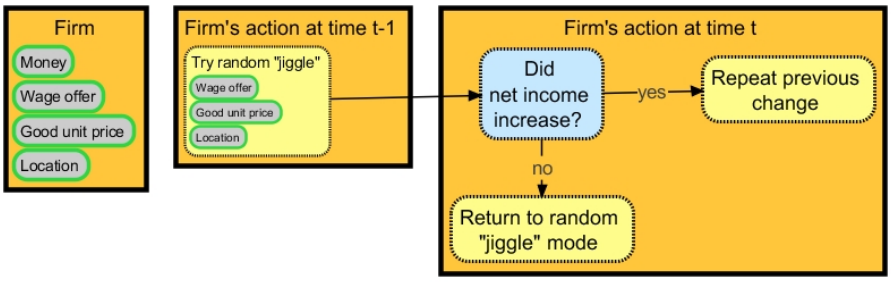


Figure 1.5: A Firm’s feedback mechanism: “jiggle” a random value, wait to see effect.

and this is a key area where I need to work through other options that might help the firm coordinate better.

Firms’ production functions

The simplest Firm production function turns a given amount of labour input into good output. For a given time period, how can this be worked out? Section **??** explored the problems of getting agents to utilise increasing returns in an agent-based time structure. The goal is to allow the model’s Firms to gain from those increasing returns: output per unit of input labour should increase as the total labour contribution rises. The problem is finding an effective way to allow that total labour quantity to change, given the discrete, asynchronous nature of decision-making.

The solution used is as follows. Two things happen: first, when a Person chooses an employer, they receive their wage on the spot, and their contribution to the Firm’s production is calculated then and there. However, they ‘leave’ a record of their contributed time with their chosen employer until it is their turn to act again, when they remove it and decide anew. This means that, when anyone else comes to contribute, that time is still considered as an input, as far as the production function is concerned. This allows increasing returns to occur. This approach is discussed in more depth in section **??** to see exactly what impact its assumptions have on model outcome.

1.3 Model ‘scenario’ setup

Individual model ‘scenarios’ will examine a particular aspect of the thesis issues. For each scenario, the main components will be described in table form as succinctly as possible. Where a scenario has particular idiosyncracies, these will be described below the model scenario table.

1.3.1 Model scenario description method

The *components* of model scenarios vary in a limited number of ways: table 1.2 lists these commonalities, and the following discussion explains the table entries in detail.

Component	Description
Run type	How many 'days'? Are parameter sweep runs used?
Actor type	1. 'people' and 'firms' 2. One actor type
Actor number	Either absolute number of agents or a set number per given radius or distance Reason for number being used
Actor action	e.g. find best wage, best good price, maximise net income Relates closely to -
Timing	When do actors act? What defines the end of a particular period of action?
Space-type	Dimension: point, region, line (R^1) or plane (R^2) If R^1 or R^2, does space 'wrap?' (Racetrack or torus.) Size in relation to actor 'reach'
Actor location	Initial location e.g. random, circle, line move type e.g. static, mobile, crawl, jump
Space costs	If on goods, how imposed? If MBF, how structured?
Values	Parameters: fixed in single model runs but may change between runs Variables: will vary during model runs Constants: fixed at all times
Utility form	e.g. Cobb Douglas, Engel or CES. Will tie to what firms are producing.
Production form	Returns to scale? Can also be 'none' of e.g. stock levels do not matter.

Table 1.2: List of model components

Run type

Model iterations are considered 'days'. A model run will last a set number of days. Generally, all actors will take an action at least once a day (with some exceptions where firms only respond every few days.) The particular number of days chosen will vary depending on the appropriate amount of time needed to find a useful result.

In each scenario, a single model run will be used to illustrate the basic point. Subsequently, parameter sweep runs may be used. These repeat model runs, while altering a particular parameter. See the discussion of parameters below for a consistent way of describing how they change.

Actor type

As outlined in section 1.1, Actors will be either of two types, Firms and People, or a single type (just referred to as 'Actors').

Actor number

The number of each type of Actor in a particular scenario can be set specifically, or given as a quantity per unit distance or radius, so that the total amount stays consistent regardless of the size of space used. Reasons for a chosen number will be given. The general principle is to use as small a number as possible to achieve useful insights. In situations where agent number affects outcomes in important ways, the total may be changed over a parameter sweep run. See also the discussion in section x on the use and meaning of agent number.

use and meaning of agent number: don’t think this section exists yet. I have it written up somewhere don’t I? The point: some dynamics come from agent number that we want to ignore, and take ‘as if’ they exist. E.g. certain competition dynamics. But where it’s still useful to have more than a single agent to make a modelling point.

Actor action

A basic description of the actions to be taken will be given for each scenario. Some are more common than others: people looking for the best wage and goods prices; firms changing cost settings in search of better net incomes. Action types are usually very intuitive and obvious. The complication arise in how they are implemented, and particularly in how they relate to timing within model runs.

Timing

Section **??** explained the issues around timing. Scenario tables will summarise as briefly as possible what particular structure is being used for a scenario.

Space type

should I bring over the space-type outline from chapter 2?

Actor location

This describes both Actors’ initial location starting conditions, and their relation to location change. Location may be randomised to start with either over the whole defined model space or within a certain distance or radius, or they may be set to a particular pattern, most often a line or circle (which stands in for a square or triangle if four or three actors are used.) Actors will also either be permanently fixed at this initial locations, or able to move. If mobile, their movement may a ‘crawl’ or a ‘jump’ - that is, they will move short distances from their current location, or will be able to move anywhere within a defined range. Lastly, these movements may cost them. This comes under the broader heading of space costs.

Space costs

A model scenario will state whether space costs are used, and how they are implemented. As well as any costs incurred through actor movement, two other types of space cost may be used: good delivery cost and land cost.

may also include land costs e.g. via landlord agents, but don't know if I'm including this in the write-up yet.

Values

The nature of the values used in a model scenario will be described. Table 1.1 explained the distinction made between variables, parameters and constants as regards each model. Of most importance will be those values set as constants for any particular scenario. For example, wages may be fixed.

1.4 Discussion / criticism

1.4.1 Where is the analytics?

Aim of section: map where math ideas from ch.2 actually appear in the framework. They have been described, but could do with more detail to draw out the differences between the approaches.

1.4.2 Misc

Very rough draft follows. Keeping in.

Why the above choices? Quick blarb. Firstly, utility is the one element from analytics this thesis prioritises. It is the micro-economic foundation of **Geographical Economics (GE)**, and I want to see what the consequences are for a) keeping it and b) putting it in an agent context. So the above setup allows people to optimise their utility based on a function like love of variety - taking any number of goods. There is plenty of criticism to be made. Some of those:

How is it realistic for people to 'sample' different spatial locations for their Bundles?

How it it realistic for people to know the price of all goods within a certain range, without expending any cost to get that information?

Same for the optimisation algorithm itself: putting an arbitrary number of goods through many steps to produce a single utility level. How is that cognitively realistic?

Answer: it isn't. We have to be open and strategic about the things that aren't realistic. My aim is to use micro-economics and see what the results are. I have to argue about how I can locate that in reality. How can People realistically acquire knowledge of all goods in an area in order to calculate utility from a function? Put that way it seems unrealistic. But it may stand in well enough for reality...

Firms have a very different decision structure to people: feedback over time using only internal data versus instant decision making from collected data. So Firms and People are active and passive in different ways. Firms can change their prices; People must accept the prices they face and optimise accordingly. People can quiz their surroundings to collect data and act immediately; Firms are forced to blindly attempt an action and wait to discover the consequences.

Firms could be given knowledge of the distance of People working for them or buying from them. Would this help? It's one of the things I need to work on: can actors be given enough information to produce stable, economically sensible outcomes?

This distinction is a consequence of the particular approach to utility used as the foundation of the modelling approach. (As discussed... where?) In contrast to **GE** and Fowler's agent approach, no 'objective' demand levels are available to the firm. They can only interpret the data they receive as the market changes. They could potentially be allowed to carry out 'market research': quizzing People about what they emphwould buy given infinite stock levels.

Section x

conclusion discussion on arbitrary model choices and exploring model space / sensitivity

discusses these problems in detail. To pre-empt that discussion briefly, there are clearly a vast range of model choices, and not all of them can be explored in this thesis. But while the avenues to explore are large, they are not impossibly so.

Bibliography

Varian, H. R. (2006). *Intermediate Microeconomics: A Modern Approach*, 7th international student edition Edition. W. W. Norton & Co.